SIMATS ENGINEERING

CSA08 - Python Programming

ASSIGNMENT 4

CO4

Robust Data Processing with Files, Exceptions, Modules and Packages

Total Marks: 50 | Individual Assignment

Solutions for Tasks 1-6

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository Link:
https://github.com/Probir127/CSA08-1911260319-Assignment4

Student and scope

Student Name: PROBIR SAHA SHOHOM. Register Number: 1911260319. Course: CSA08 - Python Programming. GitHub Repository Link: https://github.com/Probir127/CSA08-1911260319-Assignment4

This report covers Tasks 1-6, total 50 marks, in the supplied CO4 assignment: reliable file processing, error handling, modules and packages. All sample records are synthetic. Programs, input files, output reports, logs, source-change evidence and a reproducible runner accompany the PDF. The appendix lists the active programs and all four revised organisations; historical before snapshots are retained in the repository rather than repeated in the PDF.

Execution: 3.15.2 (v3.15.2:5e58eaad7c5, Oct 25 2026, 14:19:34) [Clang 21.0.0 (clang-2100.1.1.101)], standard library only, assertions enabled. Every task and all eight organisation variants were executed. Permission and write-failure tests use explicitly labelled fault injection; no real filesystem permissions were changed. Timings are measured observations, and comparison counts labelled as models are derived rather than instrumented inside Python's list implementation.

Prepared with Codex assistance. The student must understand and review the solution, execute it and submit the repository URL through the faculty's stated channel. Task commits record this preparation session, not a week of independent progressive work. Output PNGs are labelled renderings of actual captured stdout, not terminal screenshots. Capture any faculty-required terminal screenshots personally. Faculty invitation and final course submission have not been performed because no faculty account or submission destination was supplied.

Task 1 - File-processing design review (6 marks)

1.1 Diagnose the poor script (2 marks)

Deficiency	What can go wrong in the hostel office
Fixed Windows desktop path	Fails when moved to another user, operating system or month; the operator cannot choose a file
No with statement / explicit close	Closure depends on implementation cleanup; exceptions can leave resources open longer than intended
read() plus split of the whole file	Retains the entire text and another list of substrings before processing, increasing peak memory
int(parts[2]) for money	Rejects valid 1250.50 amounts, causing omitted students and a wrong total
No field-count validation	A damaged row raises IndexError, then disappears under the catch-all
Bare except: pass	Hides data errors, permission issues and programming defects; even interruption signals can be swallowed
Repeated report + fragments	Can repeatedly copy growing prefixes; predictable join-based assembly is clearer
Final newline handled by split	Creates an artificial empty final element, whose indexing error is silently discarded
No rejected-row evidence	Staff cannot identify missing bills, reconcile totals or correct the original data
No explicit encoding or CSV policy	Text may decode incorrectly; naive comma splitting mishandles a quoted comma in a name

The original open() occurs outside its try block, so a missing input file can terminate the entire script rather than be silently skipped. The bare handler inside the row loop is still defective: it hides everything raised in that loop, including unexpected programmer errors.

1.2 Repaired design and evidence (3 marks)

Task01/mess.py defines summarise(filename). It uses a binary with-open and decodes each physical line as UTF-8, enabling a decoding failure to be associated with its line number. csv.reader parses that one physical row; multiline quoted records are outside the declared one-line-per-student format. It checks exactly three fields, a numeric text register, a nonempty name and a finite nonnegative Decimal amount representing whole paise. Excess fractional paise are rejected rather than silently rounded. Decimal is created from source text, not from a float.

Known decode, CSV, numeric and decimal exceptions create entries `(line_number, raw_text, reason)` in the rejected list. Successful rows contribute to an exact total and a list of report fragments, joined once. Blank physical lines are ignored as separators; a terminal newline does not create an extra phantom record. Opening failures propagate to the caller, making unavailable input distinct from an empty valid bill file.

Hand fixture: 1250.50 + 1000.25 = **2250.75**. Three rows are rejected: a damaged field count, text 'eight' and a negative amount. The actual total and reject count match. A generated 50,000-row file with 1250.50 per row totals **62525000.00**, with zero rejects. The large file is temporary; its generator is included. Small input.csv, report.csv and rejected.json remain in Task01.

1.3 Defence in no more than 100 words (1 mark)

My version closes input files even when processing fails, reads one physical line at a time, and validates field counts and exact decimal amounts before adding them. Invalid rows are retained with line numbers and reasons rather than silently discarded. Named input parameters remove dependence on one computer's desktop path. Joining report fragments avoids repeated prefix concatenation. For 50,000 lines, input buffering stays small, although the returned report and rejected-line list still occupy memory proportional to retained content. File-access failures remain visible to the caller.

Alternative and complexity

`with open(path, encoding='utf-8') as f: data = f.read()` loads the full text; readlines() loads all lines. Both take O(B) input time for B file bytes, usually described as O(n) for n bounded-length rows. Iterating directly over the file keeps only O(L) input buffering for maximum line length L, instead of O(B) input storage. This does not make the entire supplied repaired function constant-space: it retains O(report size + rejected content), and join allocates the final report. To bound total report memory too, write accepted rows and rejects to output streams rather than returning complete lists. Known failures must still be logged, not hidden to obtain an apparently successful total.

Task 2 - Attendance file-processing design (8 marks)

2.1 Data model and rules (2 marks)

Input: UTF-8 CSV with no header, one physical row `register,YYYY-MM-DD,P-or-A` per student/day. Register numbers remain text to preserve zeroes. Dates must be real calendar dates in canonical ISO format, statuses are exactly P or A, and surrounding field whitespace is stripped. Blank attendance lines are invalid records rather than invented sessions. Files are discovered in sorted filename order with pathlib.Path.glob('*.csv'). Output directories are separate from the input directory.

The selected month is checked against the validated date prefix. A `(register,date)` set detects duplicates; the first valid in-month row in sorted-file/line order wins, and later duplicates are logged. Keeping deterministic precedence avoids counting contradictory P/A rows twice. A record from another month is rejected with an explanation.

Data structures are an accepted-record list, a dictionary of present/total counts per register, a seen-key set, an error list and file-attempt audit events. A record history is retained for the later consecutive-absence extension. Two main output files are:

• attendance_report.json: denominator policy, input_has_errors flag, all students' present/total/percentage/below_75 fields, and a separate below_75 register-number list.

• errors.csv: filename, one-based line number, exception/category, message and raw evidence. Line 0 identifies a file-level failure rather than a physical row.

Percentage is 100*present/total for accepted observed sessions. The below-75 decision uses exact integer comparison `4*present < 3*total`; a rounded display percentage does not determine eligibility. Without a roster and expected teaching-day calendar, missing files or missing student rows cannot safely be treated as absences. A damaged dataset produces a visibly qualified observed-session report, not a claim of complete monthly attendance. Students with no valid rows are unknown, not automatically 0%.

2.2 File operations (2 marks)

Read input in rb and iterate physical lines, then decode each line with utf-8. This keeps a bad byte sequence local to one line and allows later valid rows to continue. A conventional r/encoding='utf-8' reader is simpler when a whole-file decoding failure is acceptable. readline() can also stream, but iteration avoids an explicit EOF loop. read() is unnecessary for daily files.

Write fresh snapshot reports with w and explicit utf-8. The CSV writer uses newline='' to avoid platform newline duplication. Append mode a is suitable for an operational history log, but repeatedly appending monthly snapshot rows would duplicate students on reruns; therefore this implementation regenerates both monthly report files. with statements ensure closure on successful or exceptional exit. Outputs are written after aggregation; output permission/disk failures are not represented as valid empty attendance.

2.3 Processing pseudocode (3 marks)

```
ALGORITHM MONTHLY_ATTENDANCE(input_folder, month, output_folder)
BEGIN
    VERIFY input_folder is available
    files <- sorted daily CSV paths in input_folder
    accepted <- empty list
    seen <- empty set
    errors <- empty list
    FOR EACH file IN files DO
        TRY
            OPEN file for binary reading using a managed context
            FOR EACH (line_number, raw_line) IN file DO
                TRY
                    text <- DECODE raw_line as UTF-8
                    record <- PARSE and VALIDATE text
                    VERIFY record belongs to month
                    VERIFY (register, date) is not in seen
                ON invalid row or decoding failure DO
                    APPEND filename, line number and reason to errors
                OTHERWISE
                    ADD (register, date) to seen
                    APPEND record to accepted
                END TRY
            END FOR
        ON known file-access failure DO
            APPEND filename, line 0 and reason to errors
        OTHERWISE
            RECORD file-read completion
        FINALLY
            RECORD that this file attempt finished
        END TRY
    END FOR
    counts <- empty dictionary
    FOR EACH accepted record DO
        INCREMENT that student's total
        IF status = P THEN INCREMENT present END IF
    END FOR
    COMPUTE percentages and exact below-75 decisions
    WRITE attendance report and error CSV using managed contexts
END
```

This is structured pseudocode with explicit blocks; the detailed SIMATS institutional pseudocode template was not included in the supplied document, so exact compliance with an unavailable template is not claimed.

2.4 Trace and tests (1 mark)

File / row	Action	Accumulated accepted evidence
day1.csv / 1: student 0001, Oct 8, P	Accept	0001: 1 present / 1 session
day1.csv / 2: student 0002, Oct 8, P	Accept	0002: 1 present / 1 session
day1.csv / 3: broken	Reject: expected 3 fields	Counts unchanged; error includes filename and row 3
day2.csv / 1: student 0001, Oct 9, A	Accept	0001: 1 / 2, 50.00%, below 75
day2.csv / 2: student 0002, Oct 9, P	Accept	0002: 2 / 2, 100.00%
day2.csv / 3: repeat 0001, Oct 9, P	Reject duplicate	The earlier A remains; no double counting

The trace abbreviates the IDs; actual inputs use 1911260001 and 1911260002. Assertions confirm 50.0% and 100.0%, two rejects and the documented file-audit events. Complete output files are retained under Task02/results.

Alternative and complexity

Re-reading f files of l bounded-length rows once for each of s students costs O(s*f*l). One pass to collect/count costs O(f*l), with average O(1) hash updates, plus O(s log s) if the report is sorted by register. A counts-only dictionary needs O(s) state. This implementation also retains accepted records and seen register/date keys, O(n) for n sessions, and errors of size O(e); it is not a counts-only constant-memory solution. The alternative can use little per-student accumulator memory but incurs repeated file I/O and parsing. Disk-backed unique constraints/history are appropriate if retained sessions exceed memory.

Task 3 - Error-handling design (9 marks)

3.1 Failure table (3 marks)

Failure	Expected exception or behavior	Policy
Missing input folder	FileNotFoundError from the explicit folder check	Tell the caller the month cannot be discovered
Missing listed daily file	FileNotFoundError	Log line 0, attempt remaining files
Permission denied	PermissionError	Log file failure; do not invent attendance
A directory supplied as a file	IsADirectoryError on this platform	Log a file failure
Other read failure	OSError subclass	Log incomplete read and preserve visible error evidence
Invalid UTF-8 bytes	UnicodeDecodeError	Reject that physical row and continue
Wrong field count / bad status	InvalidRecord	Reject row with source evidence
Impossible calendar date	ValueError wrapped as InvalidRecord with a cause	Reject row and preserve cause chain
Naive indexing of missing fields	IndexError	Prevent by validating length before indexing
Malformed CSV quoting	csv.Error wrapped as InvalidRecord	Reject the physical row
Empty valid file	No exception; zero iterations	Do not manufacture records; unknown students remain unknown
No daily files in an existing folder	NoInputFiles evidence record	Flag the report as incomplete/empty input
Output directory/write failure	OSError / PermissionError	Expose failure; a missing report is not a successful run

There is no built-in 'empty file exception' during normal iteration. EOFError is associated with operations expecting input, not an ordinary for-loop over an empty file. Decoding may fail only when non-ASCII bytes are actually incompatible with UTF-8; an ASCII-only file in another nominal encoding may decode successfully.

3.2 Custom exception and chaining (2 marks)

The robust reader defines InvalidRecord(ValueError). parse_line raises it where row rules are checked; read_files catches it at the row boundary. A calendar parse uses `raise InvalidRecord('invalid calendar date') from error`, retaining the original ValueError in __cause__. The executed February 30 test verifies that cause. File errors are kept separate from row errors so that an unavailable file does not look like one malformed student.

3.3 Complete try and required TODO program (3 marks)

The file-level try in read_files contains a with-open. Specific except clauses handle file-access failures, else records READ_COMPLETED when file iteration finishes normally, and finally records ATTEMPT_FINISHED for every attempt, even failed opening. Row-level try/except/else appends only validated records. with closes the actual file; finally is used for audit evidence, not redundant manual closing. READ_COMPLETED means iteration completed, not that every row was valid.

Task03/complete_code.py preserves the supplied structure and implements its two TODOs. Its deliberately narrower teaching parser checks exactly three fields and P/A, while the Task02 parser adds identity/date checks. Its exact output is:

```
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
```

The separate enhanced summarise in errors_demo.py handles a nonexistent path and returns `(None,None,[(0,'file not found')])`. None distinguishes unknown counts from a successfully read file containing zero P/A records. The original expected-output exercise is not changed to hide this distinction.

Executed tests cover a real missing folder/file, an empty file, an incompatible byte line, a duplicate row, the date-cause chain and finally audit events. Permission denial is injected with unittest.mock.patch on Path.open and is labelled as simulated evidence. No claim is made that actual machine permissions were changed or that every operating-system failure was physically reproduced.

3.4 Misuse experiment (1 mark)

The deliberate typo is `present += presnt`. A broad `except Exception: pass` catches its NameError and silently returns zero despite a valid present record. Replacing that handler with `except InvalidRecord` lets NameError escape; the demonstration catches it only outside the experiment to prove the defect was visible. This is why 'the program did not crash' does not establish correctness. A bare except is even broader than except Exception because it also catches BaseException subclasses such as KeyboardInterrupt and SystemExit.

Alternative: check first versus raise/catch

An LBYL version could split fields, test len(parts)==3, validate each field with if statements, and return `(record,None)` or `(None,message)` without raising for expected invalid rows. The EAFP version raises InvalidRecord and catches it once at the row boundary. Both scan O(L) characters per bounded row; exception creation/unwinding adds a constant-factor cost on rejected rows. When failures are rare, the exceptional path runs rarely; when failures are common, explicit validation results can avoid frequent exceptions. No timings comparing these styles were measured here, so a speed winner is not asserted. Range/format checks cannot eliminate the need to catch actual file-opening errors: filesystem state can change between a pre-check and open.

Task 4 - File and exception forensics (7 marks)

4.1 Locate defects (2 marks)

Opening returns.log in w truncates it before each day's writes, erasing earlier days. write(b) adds no newline, so book identifiers run together. If write raises, the close call after the loop is skipped. The object may eventually be cleaned up by a particular interpreter, but no deterministic closure is expressed. The input open(...).read() also lacks a managed context. Only FileNotFoundError is caught; write permissions, disk faults and decoding errors escape, and even 'missing' is only printed rather than saved persistently.

4.2 Construct evidence (2 marks)

The actual three-day broken-program test uses two books per day: A/B, C/D and E/F. After each day, the log is respectively 'AB', 'CD' and 'EF'. This demonstrates both truncation and missing separators. The final broken log contains only the last day.

A retained FailingFile test object raises OSError on write and tracks its closed flag. After broken.save_returns it is False; after the with-based repaired function it is True. This is explicitly write-failure injection, not a real disk-full event. It demonstrates skipped cleanup without relying on nondeterministic garbage collection.

4.3 Repair and observed outputs (2 marks)

Task04/repaired.py opens the accumulated return log in a with statement using a and UTF-8, appends a newline for each nonblank book record, and reads daily files within their own with context. Known I/O/Unicode failures are appended as JSON lines to a separate error log; failure to write that log is reported to stderr rather than hidden. Unexpected programmer exceptions are not broadly suppressed.

After all three repaired days, returns_fixed.log contains six separate lines A through F. A missing fourth file returns False and records FileNotFoundError in errors.jsonl. Both outputs are saved in Task04. The sample day files and the reproducible temporary-file test are also included.

Append mode preserves earlier bytes, but does not make a whole day transactional. A failure midway through a day can leave a partial append; retrying without record IDs can create duplicates. Audit/transaction requirements beyond this assignment would need an idempotent design. The evidence here proves the specified append/newline/closure repairs, not crash-proof multi-file transactions.

4.4 Two prevention rules and alternative (1 mark)

1. Choose file mode from the data lifecycle: a for an accumulating log, w for an intentionally replaced snapshot; always define record delimiters and encoding. 2. Use with for every handle and record specific failures with source context; never use catch-all pass as error handling.

A manual alternative initializes handles to None, opens them inside try, and closes each successfully opened handle in finally. If closing one handle can fail, nested finally blocks are needed to still attempt closing the others. With statements encode this reliable structure compactly. Both correct styles have the same O(B) reading/writing time and O(L) streaming input memory for B bytes and longest line L, excluding retained logs/errors. The original save function can leave its one log handle unclosed on failure; the process also has an unmanaged input handle. Correct with/manual-finally versions have zero handles intentionally left open after a handled failure. During normal repaired processing, up to two handles are open together: daily input and output log. These closure guarantees assume the context manager/close implementation itself behaves normally.

Task 5 - One utility organised four ways (7 marks)

5.1 Four working organisations (3 marks)

Task05/organisations contains self-contained before and after source snapshots for each layout:

```
single/after/main.py                # reading, statistics, writing, entry point
module/after/utility.py             # all utility functions
module/after/main.py                # imports utility functions
three_modules/after/reader.py
three_modules/after/stats.py
three_modules/after/report.py
three_modules/after/main.py
package/after/attendance/__init__.py
package/after/attendance/reader.py
package/after/attendance/stats.py
package/after/attendance/report.py
package/after/main.py
```

The literal Python names are __init__.py and `if __name__ == '__main__':`, with double underscores. The source document's unadorned forms are not valid substitutes. Each main script runs only under the name guard; importing the utility functions does not read data or write reports automatically. The layouts share the same teaching file format and algorithms, copied into their own organisation, so none depends on another variant at runtime.

Task02 remains the fuller physical-line decoding/custom-exception processor. Task05's four equivalent readers focus on the organisation experiment: their file-level Unicode exception policy reports an unreadable file rather than recovering each undecodable line. This is a disclosed interface-policy difference from Task02, not a claim that reorganising code alone changed exception behavior.

5.2 Changed requirement and measured file changes (2 marks)

The added report lists students with A on three consecutive calendar dates. Missing dates break a streak; three absent rows separated by gaps do not qualify. P resets it. Dates are sorted per student, and duplicate register/day rows were rejected during reading. Tests use 1911260001 with A on Oct 1,2,3, then P; 0002 has A,A,P,P; 0003 has A on Oct 1,3,4. Only 1911260001 appears in consecutive_absent.json. All eight layouts produce identical original percentage reports; all four after layouts produce that same added absence list.

Organisation	Changed source files	Count
single	main.py	1

Organisation	Changed source files	Count
module	main.py, utility.py	2
three_modules	main.py, report.py, stats.py	3
package	attendance/report.py, attendance/stats.py, main.py	3

These counts were obtained by comparing actual .py file contents in before/after snapshots, excluding generated reports and bytecode; change_evidence.json records the names. They are not reconstructed claims about historical commits. The module's two new functions live in utility.py and main adds calls; the split layouts add computation in stats, output in report, and orchestration in main. reader and package __init__ do not change.

5.3 Compare and explain imports (2 marks)

Layout	Readability / reuse / testing	Naming and maintenance
One script	Easy to follow when tiny; direct function tests possible but importing the whole file loads every definition	One namespace; unrelated concerns grow together
One utility module + main	Reusable functions and a small entry point; tests import utility	Generic utility name can collide; independent concerns still share one file
Three modules in one folder	Reader, stats and output tests are easier to target	Generic reader/stats/report names may collide on a broad search path
attendance package	Explicit domain namespace, relative structure and reusable submodules	More small files; clearer ownership and import boundaries

`import attendance.stats` binds attendance, and functions are addressed as attendance.stats.percentage. `from attendance import stats` binds the submodule as stats. `from attendance.stats import percentage` binds the function directly. The last form is concise but can shadow another percentage function in the caller; qualified names communicate origin.

For these small files, any layout processes the same n records in O(n) aggregation time. Sorting student histories for the extension costs up to O(n log n), or less if input order can be trusted. Import cost is one-time module location/loading/top-level execution, proportional to loaded code and top-level work, not to the number of input rows unless processing wrongly occurs during import. A package creates some additional module/namespace objects. Normal imports cache modules in sys.modules within one process; separate processes load their own copies. No import-overhead timing was measured, so an invented performance number is not supplied.

A package does not automatically mean fewer changed files: this extension changed three files there versus one in the monolith. The benefit is a smaller, explicit responsibility per change, reuse and targeted testing. For a short classroom demonstration a script is enough; for the evolving attendance utility I recommend the package with a small guarded entry point.

Task 6 - Fest registration design challenge (13 marks)

Stage 1: requirements (2 marks)

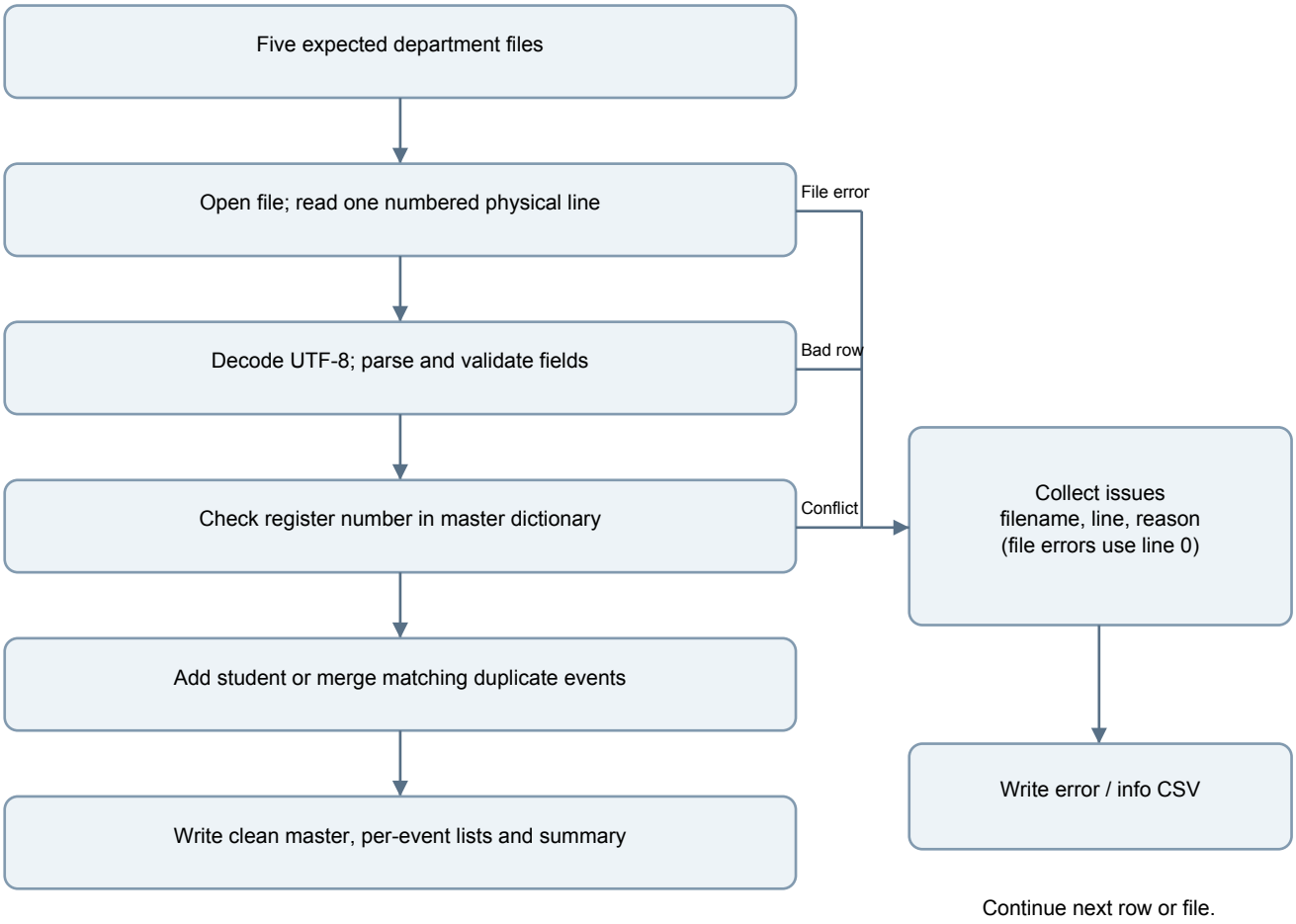
Expected input files are CSE.csv, ECE.csv, EEE.csv, MECH.csv and CIVIL.csv. Each no-header physical CSV row has register, name, department, email and a semicolon-separated event field. Names containing a comma can be quoted; multiline quoted records are outside the one-line contract. The department must match the expected source department. IDs are nonempty ASCII numeric text; no unsupported fixed digit length is invented.

The demonstrator configures Dance, Music, Drama, Quiz and Art as its allowed event catalogue. At least one known event is required. Duplicate event names collapse into a set. Email validation is a documented ASCII practical subset,

implemented with string methods: exactly one @, nonempty local part, no whitespace, no leading/trailing/consecutive local dots, permitted local characters, a domain with nonempty labels and a plausible alphabetic final label. It is not full RFC/international email validation or proof that the mailbox exists. Email case-folding is an explicit fixture policy.

Testable outputs are one unique-student master.csv, one CSV per configured event (including header-only empty events), summary.json, and errors.csv with source file/line/type/message. Every bad row is rejected and recorded while later valid rows continue. A missing department is recorded at line 0 and remaining departments continue. File output failures are exposed as failures, rather than falsely satisfying an impossible 'never stop under any circumstances' promise. The assignment's guarantee is specifically bad-line/missing-input tolerance.

Stage 2: processing flow (2 marks)



The diagram shows where validation and duplicate resolution occur. Rejection returns control to the next physical line; a file-open failure advances to the next expected department.

Stage 3: critical decisions (2 marks)

Use the master dictionary's register keys as the seen set, providing average O(1) membership without maintaining duplicate state twice. The first valid row owns the student's identity. A later row with the same ID and matching normalized name, department and email merges event choices and creates a DUPLICATE_MERGED information record. A conflicting identity is rejected and logged instead of silently replacing the first student's details. Invalid rows do not reserve their ID, so a later corrected valid row can be accepted.

Catch InvalidRecord and UnicodeDecodeError per physical row. Catch OSError around each file, including missing/permission failures. The writer manages its own with contexts, but output failures propagate; no broad except

Exception/pass masks programmer bugs. Only configured event labels generate filenames, preventing raw input from becoming arbitrary output paths. Report generation uses w because these are deterministic snapshots; the library's historical return log correctly used a for a different lifecycle.

Stage 4: package and pseudocode (4 marks)

```
Task06/
  demo.py
  duplicate_benchmark.py
  inputs/                         # four files; CIVIL intentionally missing
  fest/
    __init__.py
    validation.py                # InvalidRecord, valid_email, parse
    reader.py                    # read_registrations
    report.py                    # write_reports
    pipeline.py                  # run
  results/                        # master, five event CSVs, summary, errors
```

```
ALGORITHM FEST_REGISTRATION(input_folder, output_folder)
BEGIN
    master <- empty dictionary
    issues <- empty list
    duplicates <- 0
    FOR EACH expected department DO
        TRY
            OPEN its file in a managed binary context
            FOR EACH numbered physical line DO
                TRY
                    DECODE UTF-8 and PARSE the five CSV fields
                    VALIDATE ID, name, department, email and event choices
                    IF ID already exists in master THEN
                        IF identity fields agree THEN
                            UNION event choices into the existing entry
                            INCREMENT duplicates; RECORD duplicate information
                        ELSE
                            RAISE a conflicting-identity record error
                        END IF
                    ELSE
                        master[ID] <- clean record
                    END IF
                ON record or decoding error DO
                    RECORD department filename, line and reason
                END TRY
            END FOR
        ON file-access error DO
            RECORD filename, line 0 and reason
        END TRY
    END FOR
    WRITE sorted clean master CSV in a managed UTF-8 context
    FOR EACH configured event DO
        WRITE participating master records to that event's CSV
    END FOR
    WRITE error/information CSV and summary JSON
    RETURN summary
END
```

A failing row never enters the clean master. Event lists are derived from the deduplicated master, so an ID appears once per chosen event. CSV uses newline='' and Unicode encoding explicitly. Output isn't claimed to be an atomic all-files transaction; for crash-consistent production reports, temporary-file publication or transactional storage would be an additional requirement.

Stage 5: demonstration and evidence (3 marks)

Input / condition	Expected	Actual
Four clean students across supplied departments	Four master entries	PASS
Repeat 1911260001 with same identity and new event	One student, Dance plus Quiz, duplicate info logged	PASS
Invalid email	Reject line and continue	PASS
Damaged two-field row	Reject line with field-count reason	PASS
Same ID with conflicting identity	Reject conflicting row, retain original	PASS
Missing CIVIL.csv	One file error; other four departments processed	PASS
Invalid UTF-8 line followed by valid row	Reject first row, accept the later student	PASS
Empty Art event	Header-only art.csv exists	PASS

Main fixture summary: **4 unique students, 1 merged duplicate, 3 rejected rows, 1 missing-file error**. Event counts are Dance=1, Music=2, Drama=1, Quiz=1, Art=0. Every corresponding output file is retained in Task06/results. A separate temporary fixture verifies that an undecodable line does not prevent a later valid registration and that four other missing departments are all recorded. Email valid/invalid edge cases are asserted separately.

Alternative duplicate detection and complexity

A list alternative maintains previous register numbers and uses `if register not in previous: previous.append(register)`. For n distinct IDs, the ith lookup compares with i earlier entries, giving n(n-1)/2 comparisons and O(n^2) total time. A set or dictionary makes n expected constant-time membership operations, giving average O(n) duplicate detection; both still retain O(n) IDs. Matching duplicate records also requires comparing/merging their fields, so total byte/field size matters. This is not worst-case O(1) hashing under adversarial collisions.

Unique IDs	List equality-check model	Set memberships	List / set elapsed (ms)
1000	499500	1000	2.1531 / 0.0423
10000	49995000	10000	218.5295 / 0.4920

The equality-check counts are derived for the unique-ID benchmark; timers measure the actual list and set implementations. They are not instrumented counts inside Python's list code. The dictionary in the real pipeline provides both duplicate membership and access to the master record. Sorting n master records adds O(n log n), and scanning them for each of e events adds O(en); with the fixed five-event catalogue, this is O(n). Overall input processing is expected linear in bytes/fields plus event merges, with O(n) student state and O(r) retained issue records. Keeping every issue in memory means the program is not constant-space. Large deployments can stream rejects and use durable unique constraints, but those are separate design changes.

Reproduction and submission

Run `python3 tools/run_all.py` from the repository root, with assertions enabled. It executes every task, the required completed exercise, both error demonstrations, all eight Task05 before/after variants, the fest validation tests and the duplicate benchmark. Source inputs, clean outputs, logs, change counts and execution metadata remain available for faculty verification. The large mess input and injected failure cases are generated locally by their scripts, not fabricated external records.

The report's measured values correspond to the captured run. Re-running may change timings and process-specific temporary paths. A later report should use the corresponding later measurements. Review the assumptions, capture faculty-required terminal screenshots and submit the repository link through the faculty's chosen channel.

Appendix - Active Python source

Each listing is labeled with its task folder and filename. Task05 before snapshots are in the repository; all four revised versions are included here. Save related files in the shown folders. The companion ZIP also includes sample inputs and captured execution output. Long source lines are visually wrapped in this PDF; use the files in the ZIP when running the programs.

Task01/mess.py

```
"""Exact money, physical-line CSV validation and a retained reject list."""
import csv,json,tempfile,io
from pathlib import Path
from decimal import Decimal,InvalidOperation
def summarise(filename):
    """Return total, joined report and (line, raw text, reason) rejects."""
    total=Decimal('0.00');parts=[];rejected=[]
    with open(filename,'rb') as stream:
        for number,raw in enumerate(stream,1):
            try:
                text=raw.decode('utf-8').strip()
                if not text:continue
                fields=next(csv.reader([text],strict=True))
                if len(fields)!=3:raise ValueError('expected register,name,amount')
                register,name,amount_text=[field.strip() for field in fields]
                if not register.isascii() or not register.isdigit() or not name:raise
                    ValueError('invalid identity')
                amount=Decimal(amount_text)
                if not amount.is_finite() or amount<0:raise ValueError('invalid amount')
                rounded=amount.quantize(Decimal('0.01'))
                if amount!=rounded:raise ValueError('amount has fractional paise')
            except (UnicodeDecodeError,csv.Error,ValueError,InvalidOperation) as error:
                rejected.append((number,raw.decode('utf-8',errors='replace').rstrip(),str(error)))
                continue
            total+=rounded
            buffer=io.StringIO(newline='')
            csv.writer(buffer,lineterminator='\n').writerow((register,name,f'{rounded:.2f}'))
            parts.append(buffer.getvalue())
    return total,''.join(parts),rejected
if __name__=='__main__':
    base=Path(__file__).parent;total,report,rejects=summarise(base/'input.csv')
    assert total==Decimal('2250.75') and len(rejects)==3
    (base/'report.csv').write_text(report)
    (base/'rejected.json').write_text(json.dumps(rejects,indent=2))
    print(report,end='');print('Total:',total,'Rejects:',rejects)
    with tempfile.TemporaryDirectory() as folder:
        path=Path(folder)/'large.csv'
        with path.open('w',encoding='utf-8') as stream:
            for i in range(50000):stream.write(f'{i:010d},Student {i},1250.50\n')
        large,_,errors=summarise(path)
        assert large==Decimal('62525000.00') and errors==[]
        print('50000-line total:',large,'PASS')
        with tempfile.TemporaryDirectory() as folder:
            quoted=Path(folder)/'quoted.csv';quoted.write_text('1911260001,"Student, A",10.00\n')
            value,text,errors=summarise(quoted)
            assert value==Decimal('10.00') and errors==[]
            assert next(csv.reader(text.splitlines()))==['1911260001','Student, A','10.00']
            print('Quoted-comma name parses and writes as valid CSV PASS')
```

Task02/attendance_core.py

```
"""Attendance model: one valid record per register/date, first valid entry wins."""
import csv,json
from pathlib import Path
from datetime import date
class InvalidRecord(ValueError):
    """An attendance row violates the declared file format."""
def parse_line(line):
    """Parse register, strict ISO date, P/A; retain original failures as causes."""
    try:fields=next(csv.reader([line.strip()],strict=True))
    except csv.Error as error:raise InvalidRecord('invalid CSV') from error
    if len(fields)!=3:raise InvalidRecord(f'expected 3 fields, found {len(fields)}')
    register,day,status=[field.strip() for field in fields]
    if not register.isascii() or not register.isdigit():raise InvalidRecord('invalid register
        number')
    try:parsed=date.fromisoformat(day)
    except ValueError as error:raise InvalidRecord('invalid calendar date') from error
    if parsed.isoformat()!=day:raise InvalidRecord('date must use YYYY-MM-DD')
    if status not in ('P','A'):raise InvalidRecord(f'invalid status {status!r}')
    return register,day,status
def read_files(paths,month=None):
    """Stream binary lines, catch bad rows independently and audit every file attempt."""
    records=[];errors=[];audit=[];seen=set()
    for path in paths:
        path=Path(path)
        try:
            with path.open('rb') as stream:
                for number,raw in enumerate(stream,1):
                    try:
                        record=parse_line(raw.decode('utf-8'))
                        if month and record[1][:7]!=month:raise InvalidRecord('outside requested
                    month')
                        key=record[:2]
                        if key in seen:raise InvalidRecord('duplicate register/date')
                    except (InvalidRecord,UnicodeDecodeError) as error:
                        errors.append(dict(file=path.name,line=number,type=type(error).__name__,mes
                    sage=str(error),raw=raw.decode('utf-8',errors='replace').rstrip()))
                    else:
                        seen.add(key);records.append(record)
        except (FileNotFoundError,PermissionError,IsADirectoryError) as error:

                errors.append(dict(file=path.name,line=0,type=type(error).__name__,message=str(erro
                r),raw=''))
        except OSError as error:

                errors.append(dict(file=path.name,line=0,type=type(error).__name__,message=str(erro
                r),raw=''))
        else:audit.append((path.name,'READ_COMPLETED'))
        finally:audit.append((path.name,'ATTEMPT_FINISHED'))
    return records,errors,audit
def statistics(records):
    """Compute percentages over accepted observed sessions, never invent absences."""
    counts={}
    for register,_,status in records:
        value=counts.setdefault(register,{'present':0,'total':0})
        value['present']+=status=='P';value['total']+=1
    return [dict(register=register,**value,percentage=round(100*value['present']/value['total'],2),
        below_75=4*value['present']<3*value['total']) for register,value in sorted(counts.items())]
def write_reports(rows,errors,outdir):
    """Write two fresh report files; output failures propagate explicitly."""
    outdir=Path(outdir);outdir.mkdir(parents=True,exist_ok=True)
    report={'denominator':'accepted observed sessions only','input_has_errors':bool(errors),
            'students':rows,'below_75':[r['register'] for r in rows if r['below_75']]}
    with (outdir/'attendance_report.json').open('w',encoding='utf-8') as
        stream:json.dump(report,stream,indent=2)
    with (outdir/'errors.csv').open('w',encoding='utf-8',newline='') as stream:
        writer=csv.DictWriter(stream,fieldnames=['file','line','type','message','raw']);writer.writ
            eheader();writer.writerows(errors)
def process_month(folder,month,outdir):
    """Discover daily CSVs and create a visibly qualified monthly statement."""
    folder=Path(folder)
    if not folder.is_dir():raise FileNotFoundError(f'Input folder unavailable: {folder}')
    records,errors,audit=read_files(sorted(folder.glob('*.csv')),month)
    if not list(folder.glob('*.csv')):errors.append(dict(file=str(folder),line=0,type='NoInputFiles
        ',message='no daily files found',raw=''))
    rows=statistics(records);write_reports(rows,errors,outdir)
    return rows,errors,audit
```

Task02/demo.py

```
"""Monthly attendance example and hand trace assertions."""
from attendance_core import process_month
from pathlib import Path
if __name__=='__main__':

        base=Path(__file__).parent;rows,errors,audit=process_month(base/'inputs','2026-
        10',base/'results')
    by_id={r['register']:r for r in rows}
    assert by_id['1911260001']['percentage']==50.0
    assert by_id['1911260002']['percentage']==100.0
    assert len(errors)==2
    print('Students:',rows);print('Rejected rows:',errors);print('Audit:',audit)
    print('Observed-session percentages and damaged/duplicate row checks PASS')
```

Task03/complete_code.py

```
import os
import tempfile
class InvalidRecord(Exception):
    """Raised when a line of the attendance file is not in the expected format."""
def parse_line(line):
    """'192311001,2026-10-08,P' -> ('192311001', '2026-10-08', 'P')"""
    parts = line.strip().split(",")
    if len(parts)!=3:raise InvalidRecord(f'expected 3 fields, found {len(parts)}')
    if parts[2] not in ('P','A'):raise InvalidRecord(f'invalid status {parts[2]!r}')
    return tuple(parts)
def summarise(path):
    present, absent, errors = 0, 0, []
    with open(path,encoding='utf-8') as stream:
        for number,line in enumerate(stream,1):
            try:_,_,status=parse_line(line)
            except InvalidRecord as error:errors.append((number,str(error)))
            else:
                if status=='P':present+=1
                else:absent+=1
    return present, absent, errors
data = ("192311001,2026-10-08,P\n192311002,2026-10-08,A\n"
        "192311003,2026-10-08\n192311004,2026-10-08,L\n")
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False,
                                 encoding="utf-8") as tmp:
    tmp.write(data)
print(summarise(tmp.name))
os.remove(tmp.name)
```

Task03/errors_demo.py

```
"""Real malformed/missing/encoding tests and labelled permission fault injection."""
from pathlib import Path
import sys,tempfile,json
from unittest.mock import patch
sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'Task02'))
from attendance_core import read_files,parse_line,InvalidRecord,process_month
def summarise(path):
    """Extension: missing input yields unknown counts and an explicit file error."""
    from complete_code import summarise as original
    try:return original(path)
    except FileNotFoundError:return None,None,[(0,'file not found')]
def hidden_typo(lines):
    """Deliberately wrong: catch-all suppresses an actual NameError."""
    present=0
    for line in lines:
        try:
            parse_line(line)
            present+=presnt  # Intentional variable-name defect for the misuse experiment.
        except Exception:pass
    return present
if __name__=='__main__':
    evidence={}
    with tempfile.TemporaryDirectory() as folder:
        base=Path(folder);good=base/'good.csv';good.write_text('1911260001,2026-10-08,P\n')
        bad=base/'wrong_encoding.csv';bad.write_bytes(b'1911260001,2026-10-08,P\n\xff\n')
        empty=base/'empty.csv';empty.write_text('')
        records,errors,audit=read_files([good,bad,empty,base/'missing.csv'],'2026-10')
        assert len(records)==1 and {r['type'] for r in
            errors}=={'InvalidRecord','UnicodeDecodeError','FileNotFoundError'}
        assert sum(event=='ATTEMPT_FINISHED' for _,event in audit)==4
        evidence['real_file_failures']=errors;evidence['audit']=audit
        with patch.object(Path,'open',side_effect=PermissionError('injected permission denial')):
            _,faults,audit2=read_files([good])
        assert faults[0]['type']=='PermissionError' and audit2[-1][1]=='ATTEMPT_FINISHED'
        evidence['permission_fault_injection']=faults
        assert summarise(base/'missing.csv')==(None,None,[(0,'file not found')])
        try:process_month(base/'missing_folder','2026-10',base/'out')
        except FileNotFoundError:evidence['missing_folder']='FileNotFoundError'
        else:raise AssertionError('Missing folder accepted')
    try:parse_line('1911260001,2026-02-30,P')
    except InvalidRecord as error:
        assert
            isinstance(error.__cause__,ValueError);evidence['chained_cause']=type(error.__cause__).
            __name__
    assert hidden_typo(['1911260001,2026-10-08,P'])==0
    try:
        try:
            parse_line('1911260001,2026-10-08,P');value=presnt
        except InvalidRecord:pass
    except NameError:evidence['specific_handler_typo']='NameError propagated as expected'
    else:raise AssertionError('Typo hidden by specific handler')
    evidence['broad_handler_typo']='Silently returns zero; NameError is lost'
    Path(__file__).with_name('evidence.json').write_text(json.dumps(evidence,indent=2))
    print(json.dumps(evidence,indent=2));print('Failure, chaining, loop-else/finally audit and typo
         tests PASS')
```

Task04/broken.py

```
def save_returns(books):
    log = open("returns.log", "w")
    for b in books:
        log.write(b)
    log.close()
def process(day_file):
    try:
        books = open(day_file).read().split("\n")
        save_returns(books)
    except FileNotFoundError:
        print("missing")
```

Task04/demo.py

```
"""Three-day before/after evidence and deterministic write-failure injection."""
import os,tempfile,json
from pathlib import Path
from unittest.mock import patch
import broken,repaired
class FailingFile:
    """A retained test handle whose write raises; track explicit closure."""
    def __init__(self):self.closed=False
    def write(self,text):raise OSError('injected write failure')
    def close(self):self.closed=True
    def __enter__(self):return self
    def __exit__(self,*args):self.close()
if __name__=='__main__':
    base=Path(__file__).parent;original=Path.cwd();evidence={}
    with tempfile.TemporaryDirectory() as folder:
        os.chdir(folder)
        try:
            history=[]
            for day,books in enumerate([['A','B'],['C','D'],['E','F']],1):
                path=Path(f'day{day}.txt');path.write_text('\n'.join(books))
                broken.process(path);history.append(Path('returns.log').read_text())
            assert history==['AB','CD','EF'];evidence['broken_three_days']=history
            handle=FailingFile()
            with patch('builtins.open',return_value=handle):
                try:broken.save_returns(['X'])
                except OSError:pass
            assert not handle.closed;evidence['broken_handle_closed_after_failure']=handle.closed
            fixed=FailingFile()
            with patch('builtins.open',return_value=fixed):
                try:repaired.save_returns(['X'],'ignored')
                except OSError:pass
            assert fixed.closed;evidence['repaired_handle_closed_after_failure']=fixed.closed
            for day in range(1,4):assert
                repaired.process(f'day{day}.txt','fixed.log','errors.jsonl')
            assert Path('fixed.log').read_text()=='A\nB\nC\nD\nE\nF\n'
            assert not repaired.process('missing.txt','fixed.log','errors.jsonl')
            evidence['repaired_log']=Path('fixed.log').read_text()
            evidence['missing_file_logged']=json.loads(Path('errors.jsonl').read_text())['type']
            (base/'returns_fixed.log').write_text(Path('fixed.log').read_text())
            (base/'errors.jsonl').write_text(Path('errors.jsonl').read_text())
        finally:os.chdir(original)
    (base/'evidence.json').write_text(json.dumps(evidence,indent=2))
    print(json.dumps(evidence,indent=2));print('Three days, missing file and write-failure closure
        tests PASS')
```

Task04/repaired.py

```
"""Append returns with delimiters, deterministic closure and an error log."""
import json,sys
def save_returns(books,path):
    """Append one nonblank physical book record per line."""
    with open(path,'a',encoding='utf-8') as log:
        for book in books:
            if book.strip():log.write(book.rstrip('\r\n')+'\n')
def process(day_file,log_path,error_path):
    """Return False and log known I/O failures; never hide programmer exceptions."""
    try:
        with open(day_file,encoding='utf-8') as stream:
            save_returns(stream,log_path)
    except (OSError,UnicodeError) as error:
        record=dict(file=str(day_file),type=type(error).__name__,message=str(error))
        try:
            with open(error_path,'a',encoding='utf-8') as
                stream:stream.write(json.dumps(record)+'\n')
        except OSError as log_error:print('Cannot save error log:',log_error,file=sys.stderr)
        return False
    return True
```

Task05/compare.py

```
"""Execute all eight layouts and measure actual source-file change counts."""
from pathlib import Path
import subprocess,sys,json
if __name__=='__main__':
    base=Path(__file__).resolve().parent;results={};reference=None
    for layout in ['single','module','three_modules','package']:
        root=base/'organisations'/layout
        before={str(p.relative_to(root/'before')):p.read_text() for p in
            (root/'before').rglob('*.py')}
        after={str(p.relative_to(root/'after')):p.read_text() for p in
            (root/'after').rglob('*.py')}
        changed=[name for name in sorted(set(before)|set(after)) if
            before.get(name)!=after.get(name)]
        results[layout]={'changed_files':changed,'count':len(changed)}
        for version in ['before','after']:
            output=base/'results'/f'{layout}_{version}'
            subprocess.run([sys.executable,str(root/version/'main.py'),str(base/'inputs'),str(outpu
                t)],check=True)
            summary=json.loads((output/'attendance.json').read_text())
            if reference is None:reference=summary
            else:assert summary==reference
            if version=='after':assert
                json.loads((output/'consecutive_absent.json').read_text())==['1911260001']
    assert [results[k]['count'] for k in ['single','module','three_modules','package']]==[1,2,3,3]
    (base/'change_evidence.json').write_text(json.dumps(results,indent=2))
    print(json.dumps(results,indent=2))
    print('All eight organisations agree; only 1911260001 has 3 consecutive absent calendar dates
        PASS')
```

Task05/organisations/module/after/main.py

```
from utility import read_files,summarise,write_report,consecutive_absent,write_absences
import sys
if __name__=='__main__':
    records,errors=read_files(sys.argv[1])
    summary=summarise(records)
    write_report(summary,errors,sys.argv[2])
    write_absences(consecutive_absent(records),sys.argv[2])
```

Task05/organisations/module/after/utility.py

```
from pathlib import Path
from datetime import date
def read_files(folder):
    """Read daily physical rows; collect invalid and duplicate-record evidence."""
    records=[];errors=[];seen=set()
    for path in sorted(Path(folder).glob('*.csv')):
        try:
            with path.open(encoding='utf-8') as stream:
                for number,line in enumerate(stream,1):
                    try:
                        fields=line.strip().split(',')
                        if len(fields)!=3:raise ValueError('expected 3 fields')
                        register,day,status=fields
                        if not register.isdigit() or status not in ('P','A'):raise
                    ValueError('invalid record')
                        if date.fromisoformat(day).isoformat()!=day:raise ValueError('invalid
                    date')
                        if (register,day) in seen:raise ValueError('duplicate record')
                    except ValueError as error:errors.append((path.name,number,str(error)))
                    else:seen.add((register,day));records.append((register,day,status))
        except (OSError,UnicodeError) as error:errors.append((path.name,0,str(error)))
    return records,errors

def percentage(present,total):
    """Return observed-session percentage, or None when no sessions exist."""
    return round(present*100/total,2) if total else None
def summarise(records):
    """Count accepted sessions per student."""
    counts={}
    for register,day,status in records:
        counts.setdefault(register,[0,0]);counts[register][0]+=status=='P';counts[register][1]+=1
    return {r:dict(present=p,total=t,percentage=percentage(p,t)) for r,(p,t) in
        sorted(counts.items())}

def consecutive_absent(records):
    """Find three consecutive calendar dates marked A, not merely three stored rows."""
    from datetime import date,timedelta
    by_id={};result=set()
    for register,day,status in
        records:by_id.setdefault(register,[]).append((date.fromisoformat(day),status))
    for register,days in by_id.items():
        previous=None;length=0
        for day,status in sorted(days):
            if status=='A':length=length+1 if previous is not None and
                day==previous+timedelta(days=1) else 1
            else:length=0
            if length>=3:result.add(register)
            previous=day
    return sorted(result)

import json
from pathlib import Path
def write_report(summary,errors,folder):
    """Write summary and error evidence for this independently runnable layout."""
    folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
    (folder/'attendance.json').write_text(json.dumps(summary,indent=2))
    (folder/'errors.json').write_text(json.dumps(errors,indent=2))

def write_absences(registers,folder):
    """Write the added consecutive-absence report."""
    (Path(folder)/'consecutive_absent.json').write_text(json.dumps(registers,indent=2))
```

Task05/organisations/package/after/attendance/__init__.py

```
"""Attendance utility package; importing it does not run processing."""
```

Task05/organisations/package/after/attendance/reader.py

```
from pathlib import Path
from datetime import date
def read_files(folder):
    """Read daily physical rows; collect invalid and duplicate-record evidence."""
    records=[];errors=[];seen=set()
    for path in sorted(Path(folder).glob('*.csv')):
        try:
            with path.open(encoding='utf-8') as stream:
                for number,line in enumerate(stream,1):
                    try:
                        fields=line.strip().split(',')
                        if len(fields)!=3:raise ValueError('expected 3 fields')
                        register,day,status=fields
                        if not register.isdigit() or status not in ('P','A'):raise
                    ValueError('invalid record')
                        if date.fromisoformat(day).isoformat()!=day:raise ValueError('invalid
                    date')
                        if (register,day) in seen:raise ValueError('duplicate record')
                    except ValueError as error:errors.append((path.name,number,str(error)))
                    else:seen.add((register,day));records.append((register,day,status))
        except (OSError,UnicodeError) as error:errors.append((path.name,0,str(error)))
    return records,errors
```

Task05/organisations/package/after/attendance/report.py

```
import json
from pathlib import Path
def write_report(summary,errors,folder):
    """Write summary and error evidence for this independently runnable layout."""
    folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
    (folder/'attendance.json').write_text(json.dumps(summary,indent=2))
    (folder/'errors.json').write_text(json.dumps(errors,indent=2))

def write_absences(registers,folder):
    """Write the added consecutive-absence report."""
    (Path(folder)/'consecutive_absent.json').write_text(json.dumps(registers,indent=2))
```

Task05/organisations/package/after/attendance/stats.py

```
def percentage(present,total):
    """Return observed-session percentage, or None when no sessions exist."""
    return round(present*100/total,2) if total else None
def summarise(records):
    """Count accepted sessions per student."""
    counts={}
    for register,day,status in records:
        counts.setdefault(register,[0,0]);counts[register][0]+=status=='P';counts[register][1]+=1
    return {r:dict(present=p,total=t,percentage=percentage(p,t)) for r,(p,t) in
        sorted(counts.items())}

def consecutive_absent(records):
    """Find three consecutive calendar dates marked A, not merely three stored rows."""
    from datetime import date,timedelta
    by_id={};result=set()
    for register,day,status in
        records:by_id.setdefault(register,[]).append((date.fromisoformat(day),status))
    for register,days in by_id.items():
        previous=None;length=0
        for day,status in sorted(days):
            if status=='A':length=length+1 if previous is not None and
                day==previous+timedelta(days=1) else 1
            else:length=0
            if length>=3:result.add(register)
            previous=day
    return sorted(result)
```

Task05/organisations/package/after/main.py

```
from attendance.reader import read_files
from attendance.stats import summarise,consecutive_absent
from attendance.report import write_report,write_absences
import sys
if __name__=='__main__':
    records,errors=read_files(sys.argv[1])
    summary=summarise(records)
    write_report(summary,errors,sys.argv[2])
    write_absences(consecutive_absent(records),sys.argv[2])
```

Task05/organisations/single/after/main.py

```
from pathlib import Path
from datetime import date
def read_files(folder):
    """Read daily physical rows; collect invalid and duplicate-record evidence."""
    records=[];errors=[];seen=set()
    for path in sorted(Path(folder).glob('*.csv')):
        try:
            with path.open(encoding='utf-8') as stream:
                for number,line in enumerate(stream,1):
                    try:
                        fields=line.strip().split(',')
                        if len(fields)!=3:raise ValueError('expected 3 fields')
                        register,day,status=fields
                        if not register.isdigit() or status not in ('P','A'):raise
                    ValueError('invalid record')
                        if date.fromisoformat(day).isoformat()!=day:raise ValueError('invalid
                    date')
                        if (register,day) in seen:raise ValueError('duplicate record')
                    except ValueError as error:errors.append((path.name,number,str(error)))
                    else:seen.add((register,day));records.append((register,day,status))
        except (OSError,UnicodeError) as error:errors.append((path.name,0,str(error)))
    return records,errors

def percentage(present,total):
    """Return observed-session percentage, or None when no sessions exist."""
    return round(present*100/total,2) if total else None
def summarise(records):
    """Count accepted sessions per student."""
    counts={}
    for register,day,status in records:
        counts.setdefault(register,[0,0]);counts[register][0]+=status=='P';counts[register][1]+=1
    return {r:dict(present=p,total=t,percentage=percentage(p,t)) for r,(p,t) in
        sorted(counts.items())}

def consecutive_absent(records):
    """Find three consecutive calendar dates marked A, not merely three stored rows."""
    from datetime import date,timedelta
    by_id={};result=set()
    for register,day,status in
        records:by_id.setdefault(register,[]).append((date.fromisoformat(day),status))
    for register,days in by_id.items():
        previous=None;length=0
        for day,status in sorted(days):
            if status=='A':length=length+1 if previous is not None and
                day==previous+timedelta(days=1) else 1
            else:length=0
            if length>=3:result.add(register)
            previous=day
    return sorted(result)

import json
from pathlib import Path
def write_report(summary,errors,folder):
    """Write summary and error evidence for this independently runnable layout."""
    folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
    (folder/'attendance.json').write_text(json.dumps(summary,indent=2))
    (folder/'errors.json').write_text(json.dumps(errors,indent=2))

def write_absences(registers,folder):
    """Write the added consecutive-absence report."""
    (Path(folder)/'consecutive_absent.json').write_text(json.dumps(registers,indent=2))

import sys
if __name__=='__main__':
    records,errors=read_files(sys.argv[1])
    summary=summarise(records)
    write_report(summary,errors,sys.argv[2])
    write_absences(consecutive_absent(records),sys.argv[2])
```

Task05/organisations/three_modules/after/main.py

```
from reader import read_files
from stats import summarise,consecutive_absent
from report import write_report,write_absences
import sys
if __name__=='__main__':
    records,errors=read_files(sys.argv[1])
    summary=summarise(records)
    write_report(summary,errors,sys.argv[2])
    write_absences(consecutive_absent(records),sys.argv[2])
```

Task05/organisations/three_modules/after/reader.py

```
from pathlib import Path
from datetime import date
def read_files(folder):
    """Read daily physical rows; collect invalid and duplicate-record evidence."""
    records=[];errors=[];seen=set()
    for path in sorted(Path(folder).glob('*.csv')):
        try:
            with path.open(encoding='utf-8') as stream:
                for number,line in enumerate(stream,1):
                    try:
                        fields=line.strip().split(',')
                        if len(fields)!=3:raise ValueError('expected 3 fields')
                        register,day,status=fields
                        if not register.isdigit() or status not in ('P','A'):raise
                    ValueError('invalid record')
                        if date.fromisoformat(day).isoformat()!=day:raise ValueError('invalid
                    date')
                        if (register,day) in seen:raise ValueError('duplicate record')
                    except ValueError as error:errors.append((path.name,number,str(error)))
                    else:seen.add((register,day));records.append((register,day,status))
        except (OSError,UnicodeError) as error:errors.append((path.name,0,str(error)))
    return records,errors
```

Task05/organisations/three_modules/after/report.py

```
import json
from pathlib import Path
def write_report(summary,errors,folder):
    """Write summary and error evidence for this independently runnable layout."""
    folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
    (folder/'attendance.json').write_text(json.dumps(summary,indent=2))
    (folder/'errors.json').write_text(json.dumps(errors,indent=2))

def write_absences(registers,folder):
    """Write the added consecutive-absence report."""
    (Path(folder)/'consecutive_absent.json').write_text(json.dumps(registers,indent=2))
```

Task05/organisations/three_modules/after/stats.py

```
def percentage(present,total):
    """Return observed-session percentage, or None when no sessions exist."""
    return round(present*100/total,2) if total else None
def summarise(records):
    """Count accepted sessions per student."""
    counts={}
    for register,day,status in records:
        counts.setdefault(register,[0,0]);counts[register][0]+=status=='P';counts[register][1]+=1
    return {r:dict(present=p,total=t,percentage=percentage(p,t)) for r,(p,t) in
        sorted(counts.items())}

def consecutive_absent(records):
    """Find three consecutive calendar dates marked A, not merely three stored rows."""
    from datetime import date,timedelta
    by_id={};result=set()
    for register,day,status in
        records:by_id.setdefault(register,[]).append((date.fromisoformat(day),status))
    for register,days in by_id.items():
        previous=None;length=0
        for day,status in sorted(days):
            if status=='A':length=length+1 if previous is not None and
                day==previous+timedelta(days=1) else 1
            else:length=0
            if length>=3:result.add(register)
            previous=day
    return sorted(result)
```

Task06/demo.py

```
"""Verify clean, duplicate, invalid, damaged, missing and decoding-failure inputs."""
from pathlib import Path
import csv,json,tempfile
from fest.pipeline import run
from fest.validation import valid_email
if __name__=='__main__':
    base=Path(__file__).parent;summary=run(base/'inputs',base/'results')
    assert summary==dict(unique_students=4,duplicates_merged=1,rejected_rows=3,file_errors=1,
                         participants={'Dance':1,'Music':2,'Drama':1,'Quiz':1,'Art':0})
    with (base/'results/master.csv').open() as stream:rows=list(csv.DictReader(stream))
    assert len(rows)==4 and rows[0]['events']=='Dance;Quiz'
    for event in ['dance','music','drama','quiz','art']:assert
        (base/'results'/f'{event}.csv').exists()
    for email in ['a@example.test','student+fest@college.test']:assert valid_email(email)
    for email in ['bad-email','a@@b.test','a@b','a@.test','a b@c.test','.a@b.test']:assert not
        valid_email(email)
    with tempfile.TemporaryDirectory() as directory:
        temp=Path(directory);(temp/'CSE.csv').write_bytes(b'\xff\n1911260099,Later
            Valid,CSE,later@example.test,Art\n')
        result=run(temp,temp/'results')
        assert result['unique_students']==1 and result['rejected_rows']==1 and
            result['file_errors']==4
        print('Wrong-encoding row rejected; following valid row processed PASS')
    print(json.dumps(summary,indent=2));print('All fest requirements and validation checks PASS')
```

Task06/duplicate_benchmark.py

```
"""Count-model and timing evidence for list searching versus a seen set."""
from time import perf_counter
from pathlib import Path
import json
if __name__=='__main__':
    rows=[]
    for n in [1000,10000]:
        records=[str(1911260000+i) for i in
            range(n)];row={'n':n,'list_equality_checks_model':n*(n-
            1)//2,'set_membership_operations':n}
        begin=perf_counter();seen=[]
        for register in records:
            if register not in seen:seen.append(register)
        row['list_seconds']=perf_counter()-begin
        begin=perf_counter();lookup=set()
        for register in records:
            if register not in lookup:lookup.add(register)
        row['set_seconds']=perf_counter()-begin
        assert set(seen)==lookup and len(lookup)==n;rows.append(row)
    Path(__file__).with_name('measurements.json').write_text(json.dumps(rows,indent=2));print(json.
        dumps(rows,indent=2))
```

Task06/fest/__init__.py

```
"""College fest registration processing package."""
```

Task06/fest/pipeline.py

```
"""Coordinate independent input and output responsibilities."""
from .reader import read_registrations
from .report import write_reports
def run(folder,outdir):
    """Bad rows/missing inputs are reported; fatal output errors are not suppressed."""
    master,issues,duplicates=read_registrations(folder)
    return write_reports(master,issues,duplicates,outdir)
```

Task06/fest/reader.py

```
"""Continue past bad physical lines and missing department files."""
from pathlib import Path
from .validation import DEPARTMENTS,parse,InvalidRecord
def read_registrations(folder):
    """Return clean master dictionary, all error/info evidence and duplicate count."""
    master={};issues=[];duplicates=0
    for department in DEPARTMENTS:
        path=Path(folder)/(department+'.csv')
        try:
            with path.open('rb') as stream:
                for number,raw in enumerate(stream,1):
                    try:
                        record=parse(raw.decode('utf-8'),department)
                        register=record['register']
                        if register in master:
                            old=master[register]
                            same=(' '.join(old['name'].casefold().split())==' 
                    '.join(record['name'].casefold().split())
                                    and old['department']==record['department'] and 
                    old['email']==record['email'])
                            if not same:raise InvalidRecord('conflicting duplicate identity')
                            old['events'].update(record['events']);duplicates+=1

                        issues.append(dict(file=path.name,line=number,type='DUPLICATE_MERGED',message='
                        events merged into existing student'))
                        else:master[register]=record
                    except (InvalidRecord,UnicodeDecodeError) as error:

                        issues.append(dict(file=path.name,line=number,type=type(error).__name__,message
                        =str(error)))
        except OSError as
            error:issues.append(dict(file=path.name,line=0,type=type(error).__name__,message=str(er
            ror)))
    return master,issues,duplicates
```

Task06/fest/report.py

```
"""Deterministic CSV output with fixed allow-listed event file names."""
import csv,json
from pathlib import Path
from .validation import EVENTS
def write_reports(master,issues,duplicates,folder):
    """Write master, one file per event, summary and error log; expose output failure."""
    folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
    rows=[dict(master[key],events=';'.join(sorted(master[key]['events']))) for key in
        sorted(master)]
    fields=['register','name','department','email','events']
    def write_csv(path,values):
        """Close a new UTF-8 CSV even if writing raises."""
        with path.open('w',encoding='utf-8',newline='') as stream:

                writer=csv.DictWriter(stream,fieldnames=fields);writer.writeheader();writer.writero
                ws(values)
    write_csv(folder/'master.csv',rows)
    for event in EVENTS:write_csv(folder/(event.lower()+'.csv'),[row for row in rows if event in
        master[row['register']]['events']])
    with (folder/'errors.csv').open('w',encoding='utf-8',newline='') as stream:
        writer=csv.DictWriter(stream,fieldnames=['file','line','type','message']);writer.writeheade
            r();writer.writerows(issues)
    summary=dict(unique_students=len(master),duplicates_merged=duplicates,
                 rejected_rows=sum(i['line']>0 and i['type']!='DUPLICATE_MERGED' for i in issues),
                 file_errors=sum(i['line']==0 for i in issues),
                 participants={event:sum(event in r['events'] for r in master.values()) for event
                    in EVENTS})
    with (folder/'summary.json').open('w',encoding='utf-8') as
        stream:json.dump(summary,stream,indent=2)
    return summary
```

Task06/fest/validation.py

```
"""String-method validation for a bounded teaching registration format."""
import csv
DEPARTMENTS=('CSE','ECE','EEE','MECH','CIVIL')
EVENTS=('Dance','Music','Drama','Quiz','Art')
class InvalidRecord(ValueError):
    """A registration row is invalid."""
def valid_email(text):
    """Practical ASCII subset, not a complete international-email RFC validator."""
    if not text.isascii() or text.count('@')!=1 or any(c.isspace() for c in text):return False
    local,domain=text.split('@')
    if not local or local.startswith('.') or local.endswith('.') or '..' in local:return False
    if not all(c.isalnum() or c in '._+-' for c in local):return False
    labels=domain.split('.')
    return (len(labels)>=2 and all(label and not label.startswith('-') and not label.endswith('-')
        and all(c.isalnum() or c=='-' for c in label) for label in labels)
            and len(labels[-1])>=2 and labels[-1].isalpha())
def parse(line,department):
    """Validate identity, expected department, email and known event names."""
    try:fields=next(csv.reader([line.strip()],strict=True))
    except csv.Error as error:raise InvalidRecord('invalid CSV') from error
    if len(fields)!=5:raise InvalidRecord(f'expected 5 fields, found {len(fields)}')
    register,name,dept,email,events=[part.strip() for part in fields]
    if not register.isascii() or not register.isdigit() or not name:raise InvalidRecord('invalid
        student identity')
    if dept!=department or dept not in DEPARTMENTS:raise InvalidRecord('department mismatch')
    if not valid_email(email):raise InvalidRecord('invalid email')
    choices={event.strip() for event in events.split(';') if event.strip()}
    if not choices or not choices.issubset(EVENTS):raise InvalidRecord('missing or unknown event')
    return dict(register=register,name=name,department=dept,email=email.casefold(),events=choices)
```

tools/run_all.py

```
"""Run CO4 examples, assertions and all before/after organisation variants."""
from pathlib import Path
import subprocess,sys,json

ROOT=Path(__file__).resolve().parents[1]
JOBS={'Task01':['mess.py'],'Task02':['demo.py'],'Task03':['complete_code.py','errors_demo.py'],
      'Task04':['demo.py'],'Task05':['compare.py'],'Task06':['demo.py','duplicate_benchmark.py']}

def main():
    """Capture actual output and stop on an unexpected program/test failure."""
    results=[]
    for task,files in JOBS.items():
        chunks=[]
        for file in files:
            relative=f'{task}/{file}'
            result=subprocess.run([sys.executable,relative],cwd=ROOT,capture_output=True,
                                  text=True,check=True,timeout=90)
            chunks.append(f'$ python3 {relative}\n'+result.stdout)
            if result.stderr:chunks.append('STDERR:\n'+result.stderr)
            results.append({'program':relative,'exit_code':result.returncode})
        (ROOT/task/'output.txt').write_text('\n'.join(chunks));print(task,'PASS',flush=True)
    (ROOT/'verification.json').write_text(json.dumps({'python':sys.version,'results':results},inden
        t=2)+'\n')
    print('All CO4 examples and assertions passed.')

if __name__=='__main__':main()
```